

Day 1 — SQL & phpMyAdmin

คอร์ส Backend เบื้องต้น: PHP + SQL (phpMyAdmin)

จัดทำโดย AtSoftMaker

Day 1 — SQL & phpMyAdmin

วันนี้เราจะทำความรู้จัก **ฐานข้อมูล (Database)** และภาษา **SQL** ที่ใช้คุยกับฐานข้อมูล โดยใช้ **phpMyAdmin** (หน้าจอบริหารฐานข้อมูลที่มาพร้อมกับ XAMPP) เป็นสนามฝึก

เป้าหมายของวันนี้: เขียน SQL จัดการข้อมูลได้คล่อง ตั้งแต่ระดับพื้นฐานจนถึง JOIN และ Transaction

จุดประสงค์การเรียนรู้

เมื่อจบ Day 1 คุณจะสามารรถ:

- เข้าใจว่า Database / Table / Row / Column / Primary Key คืออะไร
- สร้างตารางและเลือกชนิดข้อมูล (datatype) ได้เหมาะสม
- เขียน **CRUD** ได้: **INSERT** (เพิ่ม), **SELECT** (อ่าน), **UPDATE** (แก้), **DELETE** (ลบ)
- กรอง/เรียง/ค้นหาข้อมูลด้วย **WHERE**, **ORDER BY**, **LIKE**, **LIMIT**
- เข้าใจหลัก **Relational Database**: ความสัมพันธ์ 1:1 / 1:N / M:N, Anomaly, Normalization
- เชื่อมหลายตารางด้วย **JOIN** และใช้ **FOREIGN KEY** (รวมถึง **ON DELETE** และตารางกลาง M:N)
- สรุปข้อมูลด้วย **COUNT** / **SUM** / **AVG** + **GROUP BY**
- ใช้ **Transaction** (**COMMIT** / **ROLLBACK** / **SAVEPOINT**) เพื่อความถูกต้องของข้อมูล

สารบัญ Day 1 (ทำตามลำดับ)

ลำดับ	ไฟล์	ประเภท	เนื้อหา
0	00_setup_xampp.md	เตรียมตัว	เปิด XAMPP, เข้า phpMyAdmin, สร้าง Database แรก
1	01_sheet_sql_basic.md	ซีท	พื้นฐานตาราง + CRUD + WHERE/ORDER/LIKE/LIMIT
2	02_lab_sql_basic.md	Lab 1	ฝึก CRUD บนตารางสินค้า
3	03_sheet_sql_advanced.md	ซีท	Relational (1:N, M:N, Normalization), FOREIGN KEY, JOIN, Aggregate, Transaction
4	04_lab_sql_advanced.md	Lab 2	ฝึก JOIN (รวม M:N) + GROUP BY + Transaction

ไฟล์ประกอบ:

- [src/basic.sql](#) — ใช้ใน Lab 1
- [src/advanced.sql](#) — ใช้ใน Lab 2

00 · เตรียมตัว: XAMPP & phpMyAdmin

จุดประสงค์

เปิดเครื่องมือให้พร้อม และสร้างฐานข้อมูลแรกของเรา เพื่อใช้ฝึกตลอด Day 1

1. XAMPP คืออะไร?

XAMPP คือชุดโปรแกรมที่รวมทุกอย่างที่ต้องใช้ทำเว็บ Backend ไว้ในที่เดียว:

ส่วนประกอบ	หน้าที่
Apache	เว็บเซิร์ฟเวอร์ (รันไฟล์ .php)
MySQL / MariaDB	ฐานข้อมูล (เก็บข้อมูล)
PHP	ภาษาเขียนโปรแกรมฝั่งเซิร์ฟเวอร์
phpMyAdmin	หน้าจอเว็บไว้จัดการฐานข้อมูลด้วยเมาส์

Day 1 เราจะใช้แค่ **MySQL** กับ **phpMyAdmin** ส่วน Apache/PHP จะได้ใช้เต็มที่ ใน Day 2

2. เปิดใช้งาน

- เปิดโปรแกรม **XAMPP Control Panel**
- กด **Start** ที่ **Apache** และ **MySQL** ให้ขึ้นแถบสีเขียว

XAMPP Control Panel

Module	PID	Port	Actions
Apache	1234	80, 443	[Stop] (on)
MySQL	5678	3306	[Stop] (on)

↑ ทั้งสองแถวเป็นสีเขียว = พร้อมใช้งาน

ถ้า Start ไม่ขึ้น (มักเป็น Apache ติด port 80) ให้ปิดโปรแกรมที่แย่ง port เช่น Skype หรือเปลี่ยน port ของ Apache เป็น 8080

3. เข้า phpMyAdmin

เปิดเบราว์เซอร์ พิมพ์:

`http://localhost/phpmyadmin`

จะเจอหน้าจอจัดการฐานข้อมูล แถบซ้ายคือรายชื่อ Database ตรงกลางคือพื้นที่ทำงาน

แท็บสำคัญที่จะใช้บ่อย:

แท็บ	ใช้ทำอะไร
Databases	ดู/สร้างฐานข้อมูล
SQL	พิมพ์คำสั่ง SQL เอง (เราจะใช้แท็บนี้มากที่สุด)
Structure	ดูโครงสร้างตาราง (คอลัมน์, ชนิดข้อมูล)
Browse	ดูข้อมูลในตารางแบบตาราง Excel
Import / Export	นำไฟล์ <code>.sql</code> เข้า / ส่งออก

4. สร้างฐานข้อมูลแรก

เราจะสร้างฐานข้อมูลชื่อ `atsoft_day1` ไว้ฝึกวันนี้

วิธี A — ใช้เมาส์ (GUI):

- คลิกแท็บ **Databases**
- ช่อง "Database name" พิมพ์ `atsoft_day1`
- ช่อง Collation เลือก **utf8mb4_general_ci** (รองรับภาษาไทย/อีโมจิ)
- กด **Create**

วิธี B — ใช้คำสั่ง SQL (แนะนำให้ลองด้วย):

คลิกแท็บ **SQL** แล้ววางคำสั่งนี้ กด **Go**

```
CREATE DATABASE atsoft_day1
CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;
```

ทำไมต้อง utf8mb4? เพราะรองรับภาษาไทยและอีโมจิได้ครบ ถ้าเลือกผิดอาจเจอตัวอักษรเพี้ยนเป็น ???

เมื่อสร้างเสร็จ จะเห็น `atsoft_day1` โผล่ในแถบซ้าย — **คลิกเลือกมัน** ก่อนทำงานต่อทุกครั้ง (เพื่อบอกว่าเรากำลังทำงานในฐานข้อมูลนี้)

ซีท 01 · SQL พื้นฐาน (CRUD)

จุดประสงค์

เข้าใจโครงสร้างฐานข้อมูล และเขียนคำสั่ง SQL พื้นฐานเพื่อ เพิ่ม / อ่าน / แก้ / ลบ ข้อมูลได้

วิธีฝึก: เปิด phpMyAdmin → เลือกฐานข้อมูล `atsoft_day1` → แท็บ **SQL** → พิมพ์ตามตัวอย่างที่ละอัน แล้วกด **Go** ดูผลลัพธ์

1. ฐานข้อมูลหน้าตาเป็นอย่างไร

ลองนึกถึงไฟล์ Excel:

คำศัพท์	คือ	เทียบกับ Excel
Database	ฐานข้อมูล 1 ก้อน	ไฟล์ Excel 1 ไฟล์
Table (ตาราง)	ตารางเก็บข้อมูลเรื่องหนึ่ง	1 ซีท (sheet)
Column (คอลัมน์/ฟิลด์)	หัวข้อข้อมูล	หัวคอลัมน์ A, B, C
Row (แถว/เรคคอร์ด)	ข้อมูล 1 รายการ	1 แถวข้อมูล
Primary Key (PK)	คอลัมน์ที่ใช้ระบุแถวแบบไม่ซ้ำ	เลขที่/รหัสประจำตัว

ตัวอย่างตาราง `products` (สินค้า):

id (PK)	name	price	stock
1	ปากกา	15.00	100
2	สมุด	25.00	50
3	ดินสอ	8.00	200

2. ชนิดข้อมูล (Data Types) ที่ใช้บ่อย

เวลาสร้างคอลัมน์ ต้องบอกว่าจะเก็บข้อมูลชนิดไหน:

ชนิด	เก็บอะไร	ตัวอย่าง
INT	จำนวนเต็ม	1, 250, -7
DECIMAL(10,2)	ทศนิยม (เหมาะกับเงิน)	15.00, 999.50
VARCHAR(100)	ข้อความสั้น (กำหนดความยาวสูงสุด)	"ปากกา", ชื่อคน
TEXT	ข้อความยาว	รายละเอียด, หมายเหตุ
DATE	วันที่	2026-06-08
DATETIME	วันที่+เวลา	2026-06-08 14:30:00
TINYINT(1)	ค่า 0/1 (ใช้แทน true/false)	1 = ใช่, 0 = ไม่

เก็บ เงิน ใช้ DECIMAL อย่าใช้ทศนิยมลอยตัว (FLOAT) เพราะอาจปัดเศษเพี้ยน

3. สร้างตาราง (CREATE TABLE)

```
CREATE TABLE products (  
  id      INT AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(100) NOT NULL,  
  price   DECIMAL(10,2) NOT NULL DEFAULT 0,  
  stock   INT NOT NULL DEFAULT 0  
);
```

อธิบายทีละบรรทัด:

- `id INT AUTO_INCREMENT PRIMARY KEY` → คอลัมน์ id เป็นเลขเพิ่มอัตโนมัติ (1,2,3,...) และเป็นกุญแจหลัก
- `NOT NULL` → ห้ามเว้นว่าง (ต้องมีค่าเสมอ)
- `DEFAULT 0` → ถ้าไม่ใส่ค่าให้ใช้ 0

ใน phpMyAdmin คุณสร้างตารางด้วยหน้าจอ (กรอกชื่อคอลัมน์/ชนิด) ก็ได้ แต่ขอให้ลองพิมพ์ SQL เองด้วย เพราะ Day 2 ต้องใช้

4. CRUD — หัวใจของการจัดการข้อมูล

CRUD ย่อมาจาก **Create, Read, Update, Delete**

อยากทำ	คำสั่ง SQL
เพิ่มข้อมูล (Create)	INSERT
อ่านข้อมูล (Read)	SELECT
แก้ข้อมูล (Update)	UPDATE
ลบข้อมูล (Delete)	DELETE

4.1 INSERT — เพิ่มข้อมูล

```
INSERT INTO products (name, price, stock)
VALUES ('ปากกา', 15.00, 100);
```

เพิ่มหลายแถวพร้อมกัน:

```
INSERT INTO products (name, price, stock) VALUES
('สมุด', 25.00, 50),
('ดินสอ', 8.00, 200);
```

ไม่ต้องใส่ id เพราะ AUTO_INCREMENT จะเติมให้เอง

4.2 SELECT — อ่านข้อมูล

อ่านทุกคอลัมน์ ทุกแถว:

```
SELECT * FROM products;
```

เลือกเฉพาะบางคอลัมน์:

```
SELECT name, price FROM products;
```

ผลลัพธ์:

name	price
ปากกา	15.00
สมุด	25.00
ดินสอ	8.00

4.3 WHERE — กรองเฉพาะที่ต้องการ

```
-- สินค้าราคามากกว่า 10 บาท
SELECT * FROM products WHERE price > 10;

-- สินค้าชื่อ "สมุด" พอดี
SELECT * FROM products WHERE name = 'สมุด';

-- สต็อกน้อยกว่าหรือเท่ากับ 50
SELECT * FROM products WHERE stock <= 50;
```

ตัวเปรียบเทียบที่ใช้ได้: = , > , < , >= , <= , <> (ไม่เท่ากับ)

รวมหลายเงื่อนไขด้วย AND / OR :

```
-- ราคามากกว่า 10 และ สต็อกมากกว่า 50
SELECT * FROM products WHERE price > 10 AND stock > 50;
```

หาค่าในช่วงด้วย BETWEEN ... AND ... (รวมค่าขอบทั้งสองข้าง):

```
-- สินค้าราคา 10 ถึง 50 บาท
-- เทียบเท่ากับ WHERE price >= 10 AND price <= 50 แต่เขียนสั้นกว่า
SELECT * FROM products WHERE price BETWEEN 10 AND 50;
```

4.4 LIKE — ค้นหาข้อความบางส่วน

% แทน "ตัวอักขระอะไรก็ได้กี่ตัวก็ได้"

```
-- ชื่อที่ขึ้นต้นด้วย "ด"
SELECT * FROM products WHERE name LIKE 'ด%';

-- ชื่อที่มีคำว่า "มุด" อยู่ข้างใน
SELECT * FROM products WHERE name LIKE '%มุด%';
```

4.5 ORDER BY — เรียงลำดับ

-- เรียงราคาจากน้อยไปมาก (ASC = น้อย→มาก, เป็นค่าเริ่มต้น)
SELECT * FROM products ORDER BY price ASC;

-- เรียงราคาจากมากไปน้อย
SELECT * FROM products ORDER BY price DESC;

4.6 LIMIT — จำกัดจำนวนแถว

-- เอามาแค่ 2 แถวแรก
SELECT * FROM products LIMIT 2;

-- สินค้าราคาแพงสุด 3 อันดับ
SELECT * FROM products ORDER BY price DESC LIMIT 3;

4.7 UPDATE — แก้ไขข้อมูล

-- เปลี่ยนราคาปากกา (id = 1) เป็น 18 บาท
UPDATE products
SET price = 18.00
WHERE id = 1;

สำคัญมาก: อย่าลืม **WHERE** ! ถ้าลืม จะแก้ ทุกแถว ในตาราง

sql **UPDATE products SET price = 0;** -- ราคาเป็น 0 หมดทุกชิ้น!

4.8 DELETE — ลบข้อมูล

-- ลบสินค้า id = 3
DELETE FROM products WHERE id = 3;

เช่นเดียวกับ UPDATE — ลืม **WHERE** = ลบหมดทั้งตาราง

5. สรุป Cheat Sheet พื้นฐาน

```
-- CREATE
INSERT INTO ตาราง (คอลัมน์1, คอลัมน์2) VALUES (ค่า1, ค่า2);

-- READ
SELECT * FROM ตาราง WHERE เงื่อนไข ORDER BY คอลัมน์ DESC LIMIT จำนวน;

-- UPDATE
UPDATE ตาราง SET คอลัมน์ = ค่าใหม่ WHERE เงื่อนไข;

-- DELETE
DELETE FROM ตาราง WHERE เงื่อนไข;
```

กฎทอง: UPDATE และ DELETE ต้องมี WHERE เสมอ (ยกเว้นตั้งใจทำกับทุกแถวจริงๆ)

Lab 1 · ฝึก SQL พื้นฐาน (CRUD)

เป้าหมาย

ฝึกเขียน `INSERT` / `SELECT` / `UPDATE` / `DELETE` พร้อม `WHERE` , `LIKE` , `ORDER BY` , `LIMIT` บนตารางสินค้าจริง

สิ่งที่ต้องเตรียม

1. เปิด phpMyAdmin → เลือกฐานข้อมูล `atsoft_day1`
2. นำเข้าข้อมูลตั้งต้น: แท็บ **Import** → เลือกไฟล์ `sql/01_basic_seed.sql` → **Go**
(หรือเปิดไฟล์ คัดลอกเนื้อหาไปวางในแท็บ **SQL** แล้ว **Go**)
3. ตรวจสอบว่ามีตาราง `products` พร้อมข้อมูล 10 แถว

วิธีทำ Lab: อ่านโจทย์ → เขียนคำสั่ง SQL ในแท็บ SQL → กด Go → ดูผลลัพธ์
เขียนคำตอบของคุณไว้ในช่องว่างใต้แต่ละข้อ แล้วค่อยเทียบ เฉลย

ส่วนที่ 1: READ (SELECT) — อ่านและค้นหา

ข้อ 1.1 แสดงสินค้าทั้งหมด (ทุกคอลัมน์)

-- เขียนคำตอบที่นี่

ข้อ 1.2 แสดงเฉพาะคอลัมน์ `name` และ `price` ของทุกสินค้า

ข้อ 1.3 แสดงสินค้าที่อยู่ในหมวด `เครื่องดื่ม`

ข้อ 1.4 แสดงสินค้าที่ราคาสูงกว่า 20 บาท

ข้อ 1.5 แสดงสินค้าที่ **สต็อกน้อยกว่า 50** (ของใกล้หมด)

ข้อ 1.6 ค้นหาสินค้าที่ชื่อมีคำว่า น้ำ

ข้อ 1.7 แสดงสินค้าทั้งหมด เรียงตามราคาจาก มากไปน้อย

ข้อ 1.8 แสดงสินค้าที่ราคาแพงที่สุด 3 อันดับแรก

ข้อ 1.9 แสดงสินค้าที่ราคาอยู่ระหว่าง 10 ถึง 50 บาท (ใช้ BETWEEN)

ส่วนที่ 2: CREATE (INSERT) — เพิ่มข้อมูล

ข้อ 2.1 เพิ่มสินค้าใหม่ 1 ชิ้น: ชื่อ แก้วพลาสติก , หมวด บรรจุภัณฑ์ , ราคา 2.50 , สต็อก 1000

ข้อ 2.2 เพิ่มสินค้าพร้อมกัน 2 ชิ้นในคำสั่งเดียว (คิดชื่อ/ราคาเองได้)

ข้อ 2.3 SELECT ดูอีกครั้ง ว่าสินค้าที่เพิ่งเพิ่มเข้าไปอยู่ครบไหม

ส่วนที่ 3: UPDATE — แก้ไขข้อมูล

ข้อ 3.1 สินค้า หมวดนริภย ขึ้นราคาเป็น 280.00 — แก้ให้ถูกต้อง

ใช้ WHERE name = '...' หรือดู id จากตารางก่อน

ข้อ 3.2 เพิ่มสต็อกของ น้ำดื่ม 600ml อีก 200 ชิ้น (จากเดิม 500 เป็น 700)

ทำได้ 2 แบบ: ใส่ค่าใหม่ตรงๆ (`SET stock = 700`) หรือใช้ `SET stock = stock + 200`

ข้อ 3.3 ลดราคาสินค้าทุกชิ้นในหมวด อุปกรณ์ ลง 10% (ราคาใหม่ = ราคาเดิม $\times$ 0.9)

ส่วนที่ 4: DELETE — ลบข้อมูล

ข้อ 4.1 ลบสินค้าที่ชื่อ เทปขาวใส

ข้อ 4.2 ลบสินค้าทั้งหมดที่สต็อกเป็น 0 (ถ้ามี)

โจทย์ท้าทาย (ทำถ้าเหลือเวลา)

ข้อ 5.1 หาสินค้าหมวด อุปกรณ์ ที่ราคาแพงที่สุด 2 อันดับแรก

รวม `WHERE` + `ORDER BY ... DESC` + `LIMIT` เข้าด้วยกัน

ข้อ 5.2 นับว่ามีสินค้าในหมวด อุปกรณ์ กี่ชิ้น

ใช้ `SELECT COUNT(*) ...` (เป็นการ preview เนื้อหาซีทถัดไป)

เช็คความเข้าใจ

- [] เขียน SELECT + WHERE + ORDER BY + LIMIT ได้
- [] เพิ่มข้อมูลด้วย INSERT ได้ทั้งทีละแถวและหลายแถว
- [] แก้ข้อมูลด้วย UPDATE โดยใช้ WHERE ทุกครั้ง
- [] ลบข้อมูลด้วย DELETE อย่างระมัดระวัง

ซีท 03 · SQL ขั้นสูง (Relationship, JOIN, Aggregate, Transaction)

จุดประสงค์

- เข้าใจหลักการ **Relational Database**: ชนิดความสัมพันธ์ (1:1, 1:N, M:N), Anomaly, Normalization
- เชื่อมข้อมูลหลายตารางด้วย **FOREIGN KEY** (+ **ON DELETE**) และ **JOIN**
- สรุปข้อมูลด้วยฟังก์ชัน **Aggregate** + **GROUP BY**
- ใช้ **Transaction** (COMMIT / ROLLBACK / SAVEPOINT) เพื่อให้ข้อมูลถูกต้องเมื่อทำหลายคำสั่งพร้อมกัน

เตรียมข้อมูล: Import ไฟล์ `sql/02_advanced_seed.sql` เข้าฐานข้อมูล `atsoft_day1` ก่อน

1. Relational Database คืออะไร?

ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) คือฐานข้อมูลที่เก็บข้อมูลเป็น **หลายตาราง** แล้ว "เชื่อมโยง" กันด้วยรหัส แทนที่จะยึดทุกอย่างไว้ตารางเดียว — MySQL/MariaDB ที่เราใช้อยู่ก็เป็นแบบนี้

1.1 ปัญหาของการเก็บไว้ตารางเดียว

สมมติเราเก็บ "คำสั่งซื้อ" ไว้ตารางเดียว แล้วใส่ชื่อลูกค้าซ้ำๆ ทุกแถว:

order_id	customer_name	customer_phone	product	amount
1	สมชาย	081-111	น้ำดื่ม	2
2	สมชาย	081-111	สมุด	1

ปัญหานี้เรียกว่า **ความผิดปกติของข้อมูล (Data Anomaly)** มี 3 แบบ:

ชนิด	ปัญหา	ตัวอย่าง
Update Anomaly	แก้ที่เดียวไม่พอ ต้องไล่แก้ทุกแถว	สมชายเปลี่ยนเบอร์ ต้องแก้ทุกแถวที่มีสมชาย ถ้าลืมแถวใด ข้อมูลจะขัดกันเอง
Insert Anomaly	เพิ่มข้อมูลบางอย่างไม่ได้ถ้ายังไม่มีอีกอย่าง	อยากเก็บลูกค้าใหม่ที่ "ยังไม่เคยสั่งซื้อ" ทำไม่ได้ เพราะตารางนี้บังคับต้องมี product
Delete Anomaly	ลบของอย่างหนึ่ง แล้วข้อมูลอีกอย่างหายไปด้วย	ลบคำสั่งซื้อสุดท้ายของสมชาย → ข้อมูลเบอร์โทรของสมชายหายไปเลย

ทางแก้: แยกเป็น 2 ตาราง แล้วเชื่อมกันด้วย "รหัส"

ตาราง `customers`

id	name	phone
1	สมชาย	081-111

ตาราง `orders` (เก็บแค่ `customer_id` อ้างอิงลูกค้า)

id	customer_id	product	amount
1	1	น้ำดื่ม	2
2	1	สมุด	1

`customer_id` ในตาราง `orders` ที่ชี้ไปหา `id` ในตาราง `customers` เรียกว่า **FOREIGN KEY** (กุญแจนอก)

1.2 ชนิดของความสัมพันธ์ (Relationship) — 3 แบบ

แบบ	อ่านว่า	ความหมาย	ตัวอย่าง
1 : 1	One-to-One	1 แถวฝั่งนี้ จับคู่กับ 1 แถวฝั่งโน้น	1 พนักงาน มี 1 ข้อมูลเงินเดือน
1 : N	One-to-Many	1 แถวฝั่งนี้ มีได้ "หลายแถว" ฝั่งโน้น	1 ลูกค้า มีได้หลายคำสั่งซื้อ
M : N	Many-to-Many	หลายแถวฝั่งนี้ จับกับหลายแถวฝั่งโน้น	1 คำสั่งซื้อมีหลายสินค้า / 1 สินค้าอยู่ในหลายคำสั่งซื้อ

1 : N (พบบ่อยที่สุด) — customers ↔ orders

ฝั่ง "หลาย" (orders) เก็บ FOREIGN KEY ชี้กลับไปหาฝั่ง "หนึ่ง" (customers)

customers				orders		
id	name			id	customer_id	product
1	สมชาย	1	N	1	1	น้ำดื่ม
				2	1	สมุด

M : N — orders ↔ products

ความสัมพันธ์แบบนี้ใส่ FOREIGN KEY ตรงๆ ไม่ได้ ต้องสร้าง ตารางกลาง (junction table) มาคั่น
ในแล็บเราใช้ตาราง order_items (1 แถว = "สินค้า 1 รายการในคำสั่งซื้อ 1 ใบ")

orders			order_items (ตารางกลาง)			products	
id			order_id	product_id		id	name
1	1	N	1	1	N	1	น้ำดื่ม
			1	4			
			2	1			

← คำสั่งซื้อ #1 มี 2 สินค้า
← สินค้า #1 อยู่หลายคำสั่งซื้อ

สรุป: M:N = แยกออกเป็นสอง 1:N ผ่านตารางกลาง — ตารางกลางมักมีคอลัมน์เสริม เช่น quantity (จำนวน)

1.3 Normalization — จัดระเบียบตารางให้ไม่ซ้ำซ้อน

Normalization คือการ "แตกตารางใหญ่ที่ข้อมูลซ้ำซ้อน ออกเป็นตารางเล็กที่สะอาด" เพื่อกัน Anomaly
มี 3 ขั้นที่ใช้อยู่ (ทำไล่จากบนลงล่าง):

ขั้น	กฎแบบเข้าใจง่าย
1NF	แต่ละช่องเก็บ "ค่าเดียว" ห้ามยัดหลายค่าในช่องเดียว (เช่นห้ามเก็บ "ปากกา, สมุด" ในช่องเดียว)
2NF	ผ่าน 1NF + ทุกคอลัมน์ต้องขึ้นกับ Primary Key ทั้งหมด (สำคัญตอน PK เป็นหลายคอลัมน์)
3NF	ผ่าน 2NF + คอลัมน์ที่ไม่ใช่ key ต้อง ไม่ขึ้นต่อกันเอง (เช่นอย่าเก็บทั้ง <code>customer_id</code> และ <code>customer_phone</code> ในตาราง orders — เบอร์ควรอยู่ที่ customers)

ตัวอย่าง: ตารางเดิม (ไม่ผ่าน normalization)

order_id	customer_name	customer_phone	products
1	สมชาย	081-111	น้ำดื่ม, สมุด

จัดระเบียบเป็น 3 ตาราง: `customers` (ข้อมูลลูกค้า), `orders` (หัวคำสั่งซื้อ), `order_items` (รายการสินค้าในแต่ละใบ)
→ ไม่มีข้อมูลซ้ำ, แก่เบอร์ที่เดียว, เพิ่มลูกค้าที่ยังไม่สั่งซื้อได้

ในทางปฏิบัติ มักทำถึง **3NF** ก็เพียงพอ บางครั้งงานที่เน้นความเร็วในการอ่านอาจใจ "de-normalize"
(ยอมเก็บซ้ำบ้าง) เพื่อแลกกับ JOIN ที่น้อยลง — เป็นการเลือกตามสถานการณ์

2. FOREIGN KEY — สร้างความสัมพันธ์

```
CREATE TABLE customers (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  phone   VARCHAR(20)
);

CREATE TABLE orders (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  customer_id   INT NOT NULL,
  product       VARCHAR(100) NOT NULL,
  amount        INT NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

ประโยชน์: ฐานข้อมูลจะ **ปกป้องความถูกต้อง** เช่น ห้ามใส่ `customer_id` ที่ไม่มีอยู่จริงในตาราง customers
(เรียกว่า **Referential Integrity** — ความถูกต้องของการอ้างอิง)

2.1 จะเกิดอะไรเมื่อลบ "แม่" ที่มี "ลูก" อ้างอยู่?

ถ้าลองลบลูกค้า id=1 ทั้งที่ยังมี orders อ้างอิงอยู่ ฐานข้อมูลจะ **ปฏิเสธ (error)** เพื่อกันข้อมูลกำพริ้
เรากำหนดพฤติกรรมตรงนี้ได้ด้วย **ON DELETE / ON UPDATE** :

ตัวเลือก	ความหมายเมื่อ "ลบ/แก้ แถวแม่"
RESTRICT / NO ACTION	ห้าม ลบถ้ายังมีลูกอ้างอิงอยู่ (ค่าเริ่มต้น ปกติที่สุด)
CASCADE	ลบแม่แล้ว ลบลูกตามไปด้วย อัตโนมัติ
SET NULL	ลบแม่แล้ว ตั้งค่า FK ในลูกเป็น NULL (คอลัมน์ต้องอนุญาต NULL)

```
CREATE TABLE orders (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  product     VARCHAR(100) NOT NULL,
  amount      INT NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers(id)
    ON DELETE CASCADE -- ลบลูกค้า → ลบคำสั่งซื้อของเขาไปด้วย
    ON UPDATE CASCADE -- ถ้า id ลูกค้าเปลี่ยน → อัปเดต customer_id ตามให้
);
```

2.2 สร้างตารางกลางสำหรับ Many-to-Many

ความสัมพันธ์ M:N ระหว่าง **orders** กับ **products** ทำผ่านตารางกลาง **order_items**
ที่มี FOREIGN KEY สองตัว ชี้ไปทั้งสองฝั่ง:

```
CREATE TABLE order_items (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  order_id    INT NOT NULL,
  product_id  INT NOT NULL,
  quantity    INT NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

1 แถว = "สินค้า 1 รายการในคำสั่งซื้อ 1 ใบ" — คอลัมน์ **quantity** คือข้อมูลเสริมของความสัมพันธ์นั้น

3. JOIN — เชื่อมตารางตอนอ่านข้อมูล

เราอยากเห็น "ชื่อลูกค้า" คู่กับ "สินค้าที่สั่ง" ในตารางเดียว → ใช้ JOIN

3.1 INNER JOIN (เอาเฉพาะที่จับคู่กันได้)

```
SELECT orders.id, customers.name, orders.product, orders.amount
FROM orders
INNER JOIN customers ON orders.customer_id = customers.id;
```

ผลลัพธ์:

id	name	product	amount
1	สมชาย	น้ำดื่ม	2
2	สมชาย	สมุด	1

ใช้ชื่อย่อ (alias) ให้สั้นลงได้ด้วย:

```
SELECT o.id, c.name, o.product, o.amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id;
```

3.2 LEFT JOIN (เอาทุกแถวฝั่งซ้าย แม้ไม่มีคู่)

```
-- เอาลูกค้า "ทุกคน" แม้คนที่ยังไม่เคยสั่งซื้อ
SELECT c.name, o.product
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id;
```

ลูกค้าที่ยังไม่เคยสั่ง จะได้ค่า NULL ในคอลัมน์ product

3.3 RIGHT JOIN (เอาทุกแถวฝั่งขวา แม้ไม่มีคู่)

ทำงานกลับด้านกับ LEFT JOIN — เอาทุกแถวของตาราง "ฝั่งขวา" (ตารางหลัง JOIN) ให้ครบ

```
-- เอาคำสั่งซื้อ "ทุกใบ" แม้ใบที่ไม่มีข้อมูลลูกค้าผูกอยู่
SELECT c.name, o.product
FROM customers c
RIGHT JOIN orders o ON o.customer_id = c.id;
```

จริงๆ แล้ว `A LEFT JOIN B` ให้ผลเหมือน `B RIGHT JOIN A` — เลือกใช้ตัวที่อ่านเข้าใจง่ายกว่าได้เลย โดยทั่วไปนิยมใช้ LEFT JOIN มากกว่า

จำง่าย ๆ:

- INNER JOIN = เฉพาะที่ "แมตช์กันทั้งสองฝั่ง"
- LEFT JOIN = "เอาฝั่งซ้ายให้ครบ" ฝั่งขวาไม่มีก็ใส่ NULL
- RIGHT JOIN = "เอาฝั่งขวาให้ครบ" ฝั่งซ้ายไม่มีก็ใส่ NULL

3.4 JOIN หลายตาราง (เดินผ่านตารางกลาง M:N)

อยากดู "คำสั่งซื้อไปไหน มีสินค้าอะไรบ้าง ก็ขึ้น" ต้องเดินผ่าน 3 ตาราง:

`orders` → `order_items` → `products` โดย JOIN ต่อกันเป็นทอดๆ

```
SELECT
  o.id           AS order_id,
  p.name        AS product,
  oi.quantity   AS จำนวน,
  p.price       AS ราคาต่อชิ้น
FROM orders o
INNER JOIN order_items oi ON oi.order_id = o.id
INNER JOIN products p ON p.id = oi.product_id
ORDER BY o.id;
```

ผลลัพธ์ (บางส่วน):

order_id	product	จำนวน	ราคาต่อชิ้น
1	น้ำดื่ม 600ml	2	7.00
1	กล่องกระดาษ...	1	12.50
2	ชาเขียวขวด...	5	20.00

เทคนิค M:N ที่ใช้บ่อย: JOIN 3 ตารางแล้ว GROUP BY เพื่อสรุป เช่น "ยอดเงินรวมต่อ 1 คำสั่งซื้อ"
= SUM(oi.quantity * p.price) (จะได้ฝึกในแล็บ)

4. Aggregate — สรุปข้อมูล

ฟังก์ชันสรุปผล:

ฟังก์ชัน	ความหมาย
COUNT(*)	นับจำนวนแถว
SUM (คอลัมน์)	ผลรวม
AVG (คอลัมน์)	ค่าเฉลี่ย
MIN / MAX	ค่าน้อยสุด / มากสุด
STDDEV (คอลัมน์)	ส่วนเบี่ยงเบนมาตรฐาน (S.D.)
VARIANCE (คอลัมน์)	ความแปรปรวน (variance)

-- มีคำสั่งซื้อทั้งหมดกี่รายการ
SELECT COUNT(*) FROM orders;

-- รวมจำนวนสินค้าที่สั่งทั้งหมด
SELECT SUM(amount) FROM orders;

4.1 GROUP BY — สรุปแบบแยกกลุ่ม

```
-- ลูกค้าแต่ละคน สั่งสินค้ารวมกี่ชิ้น
SELECT c.name, SUM(o.amount) AS total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
GROUP BY c.name;
```

ผลลัพธ์:

name	total_amount
สมชาย	3
มานี้	5

4.2 HAVING — กรองหลังจากสรุป

WHERE กรองก่อนรวมกลุ่ม ส่วน HAVING กรอง ผลที่รวมกลุ่มแล้ว

```
-- เฉพาะลูกค้าที่สั่งรวมเกิน 3 ชิ้น
SELECT c.name, SUM(o.amount) AS total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
GROUP BY c.name
HAVING total_amount > 3;
```

4.3 ส่วนเบี่ยงเบนมาตรฐาน (S.D.) ด้วย STDDEV

STDDEV() บอกว่าข้อมูล "กระจายตัว" มากแค่ไหนรอบๆ ค่าเฉลี่ย — ค่ายิ่งมากยิ่งกระจายห่าง

```
-- ค่าเฉลี่ย, ส่วนเบี่ยงเบนมาตรฐาน และความแปรปรวน ของราคาในคำสั่งซื้อ
SELECT
  AVG(price) AS ราคาเฉลี่ย,
  STDDEV(price) AS ส่วนเบี่ยงเบนมาตรฐาน,
  VARIANCE(price) AS ความแปรปรวน
FROM orders;
```

MySQL/MariaDB ยังมี STDDEV_POP() (คิดแบบประชากรทั้งหมด) และ STDDEV_SAMP() (คิดแบบกลุ่มตัวอย่าง) ให้เลือกตามหลักสถิติ โดย STDDEV() มีค่าเท่ากับ STDDEV_POP()

4.4 ปรับข้อมูลให้เป็นมาตรฐาน (Standardization / Normalization)

ก่อนนำตัวเลขไปวิเคราะห์ต่อ มักต้อง "ปรับสเกล" ให้เทียบกันได้ มี 2 วิธีที่พบบ่อย:

1) Standardization (Z-score) = (ค่า - ค่าเฉลี่ย) ÷ ส่วนเบี่ยงเบนมาตรฐาน

อ่านได้ว่า "ค่านี้ห่างจากค่าเฉลี่ยกี่เท่าของ S.D."

```
SELECT
  product,
  price,
  (price - (SELECT AVG(price) FROM orders))
    / ((SELECT STDDEV(price) FROM orders) AS z_score
FROM orders;
```

2) Min-Max Normalization = บีบให้อยู่ในช่วง 0 ถึง 1

สูตร: $(\text{ค่า} - \text{ค่าต่ำสุด}) \div (\text{ค่าสูงสุด} - \text{ค่าต่ำสุด})$

```
SELECT
  product,
  price,
  (price - (SELECT MIN(price) FROM orders))
    / ((SELECT MAX(price) FROM orders) - (SELECT MIN(price) FROM orders)) AS normalized
FROM orders;
```

ทั้งสองสูตรเอา "ค่าสรุปของทั้งตาราง" (AVG/STDDEV/MIN/MAX) มาคำนวณกับ "ค่ารายแถว" จึงต้องดึงค่าสรุปด้วย subquery ในวงเล็บ (...)

4.5 คำนวณช่วงวันด้วย DATEDIFF()

DATEDIFF(วันที่หลัง, วันที่ก่อน) คืนจำนวน "วัน" ที่ห่างกัน เหมาะกับงานรายงาน

```
-- คำสั่งซื้อแต่ละใบ ผ่านมากี่วันแล้ว (นับถึงวันนี้)
SELECT
  product,
  order_date,
  DATEDIFF(CURDATE(), order_date) AS ผ่านมากี่วัน
FROM orders;
```

ใช้ร่วมกับ aggregate ได้ เช่น หาค่าเฉลี่ยจำนวนวันที่ผ่านมาของทุกคำสั่งซื้อ:

```
SELECT AVG(DATEDIFF(CURDATE(), order_date)) AS เฉลี่ยจำนวนวัน
FROM orders;
```

ฟังก์ชันวันที่ที่ใช้อยู่อื่นๆ: CURDATE() (วันนี้), NOW() (วันที่+เวลาปัจจุบัน), YEAR() / MONTH() / DAY() (ดึงปี/เดือน/วันออกมา)

5. Transaction — ทำหลายคำสั่งให้ "สำเร็จทั้งหมด หรือไม่ก็ยกเลิกทั้งหมด"

ปัญหาที่ Transaction แก้

สมมติ "โอนเงิน" จากบัญชี A ไป B ต้องทำ 2 ขั้นตอน:

1. ลดเงินบัญชี A
2. เพิ่มเงินบัญชี B

ถ้าทำขั้น 1 เสร็จแล้วไฟดับก่อนขั้น 2 → เงินหายไปเฉยๆ

Transaction บอกว่า "สองขั้นนี้ต้องสำเร็จคู่กัน ถ้าพังขั้นใดขั้นหนึ่ง ให้ย้อนกลับเหมือนไม่เคยทำ"

คำสั่ง

```
START TRANSACTION;           -- เริ่ม

UPDATE accounts SET balance = balance - 100 WHERE id = 1; -- ลดเงิน A
UPDATE accounts SET balance = balance + 100 WHERE id = 2; -- เพิ่มเงิน B

COMMIT;                       -- ยืนยัน บันทึกจริงทั้งหมด
```

ถ้าระหว่างทางพบว่ามีปัญหา (เช่นเงินไม่พอ) ให้ยกเลิก:

```
ROLLBACK;                    -- ย้อนทุกอย่างกลับเหมือนไม่เคยทำ
```

ACID — คุณสมบัติที่ Transaction รับประกัน (รู้ไว้พอเข้าใจ)

ตัวอักษร	ความหมายแบบสั้น
A – Atomicity	ทำสำเร็จทั้งหมด หรือไม่ทำเลย
C – Consistency	ข้อมูลยังถูกต้องตามกฎเสมอ
I – Isolation	หลายงานพร้อมกันไม่กวนกัน
D – Durability	เมื่อ COMMIT แล้วข้อมูลอยู่ถาวร แม้ไฟดับ

Transaction ใช้ได้กับตารางชนิด **InnoDB** (ค่าเริ่มต้นของ MySQL/MariaDB ปัจจุบัน)
ในงานจริง เรามักสั่ง Transaction ผ่านโค้ด PHP (จะได้เจอใน Day 2/Day 3)

5.1 ตัวอย่างจริง: สร้างคำสั่งซื้อ + ตัดสต็อกพร้อมกัน

งานขายของจริงต้องทำ 2 อย่างให้ "สำเร็จคู่กัน": บันทึกคำสั่งซื้อ และ หักสต็อกสินค้า
ถ้าบันทึกออเดอร์สำเร็จแต่หักสต็อกพลาด → ขายเกินจำนวนที่มี

START TRANSACTION;

-- 1) หักสต็อกน้ำดื่ม (product id = 1) ออก 2 ขัน

UPDATE products **SET** stock = stock - 2 **WHERE** id = 1;

-- 2) สร้างคำสั่งซื้อใหม่ของลูกค้า id = 2

INSERT INTO orders (customer_id, product, amount, price, order_date)

VALUES (2, 'น้ำดื่ม 600ml', 2, 7.00, CURDATE());

COMMIT; -- ทั้งสองอย่างถูกบันทึกพร้อมกัน

5.2 เช็กระยะก่อนตัดสินใจ COMMIT หรือ ROLLBACK

หัวใจของ Transaction คือ "ดูผลกลางทางก่อน แล้วค่อยตัดสินใจ"

เช่น โอนเงินแล้วเช็คว่ามีเงินติดลบหรือไม่:

START TRANSACTION;

UPDATE accounts **SET** balance = balance - 9999 **WHERE** id = 1;

-- ดูยอดก่อนยืนยัน

SELECT balance **FROM** accounts **WHERE** id = 1;

-- ถ้าผลออกมา "ติดลบ" (เงินไม่พอ) → สั่ง **ROLLBACK**

-- ถ้ายอดยังเป็นบวกตามปกติ → ทำ **UPDATE** ฝั่งรับต่อ แล้ว **COMMIT**

ROLLBACK; -- ในเคสนี้เงินไม่พอ จึงยกเลิกทั้งหมด ยอดกลับเป็นเดิม

ใน phpMyAdmin เราเช็คด้วยตาเอง แต่ในโค้ด PHP จะใช้ `if` ตรวจสอบแล้วเลือก commit/rollback อัตโนมัติ

5.3 SAVEPOINT — ย้อนกลับ "บางส่วน" ของ Transaction

บางที Transaction ยาวๆ เราอยากย้อนแค่บางขั้น ไม่ต้องทิ้งทั้งก้อน ใช้ **SAVEPOINT** ตั้ง "จุดเซฟ":

START TRANSACTION;

UPDATE accounts **SET** balance = balance - 100 **WHERE** id = 1;

SAVEPOINT after_withdraw; -- ตั้งจุดเซฟไว้ตรงนี้

UPDATE accounts **SET** balance = balance + 100 **WHERE** id = 2;

-- เปลี่ยนใจ ไม่อยากให้ขั้นที่ 2 เกิดขึ้น

ROLLBACK TO after_withdraw; -- ย้อนกลับมาที่จุดเซฟ (ขั้น 1 ยังอยู่)

COMMIT; -- ยืนยันเฉพาะขั้นที่ 1

5.4 AUTOCOMMIT — ทำไมปกติพิมพ์ทีละคำสั่งถึง "บันทึกทันที"

ปกติ MySQL เปิด **AUTOCOMMIT** ไว้ คือทุกคำสั่งที่รันเดี่ยวๆ จะ **COMMIT** ให้ทันทีอัตโนมัติ

START TRANSACTION; คือการ "ปิด autocommit ชั่วคราว" จนกว่าจะ **COMMIT** หรือ **ROLLBACK** เอง

```
SET autocommit = 0;    -- ปิด: ตั้งแต่นี้ต้อง COMMIT เองทุกครั้ง
-- ... คำสั่งต่างๆ ...
COMMIT;
SET autocommit = 1;    -- เปิดกลับเป็นค่าเริ่มต้น
```

5.5 พรีวิว: Transaction ใน PHP (PDO) — เจอจริงใน Day 2

ในโค้ดจริง เราคุม commit/rollback ด้วยเงื่อนไข try / catch :

```
try {
    $pdo->beginTransaction();           // = START TRANSACTION
    $pdo->prepare('UPDATE accounts SET balance = balance - ? WHERE id = ?')
    ->execute([100, 1]);
    $pdo->prepare('UPDATE accounts SET balance = balance + ? WHERE id = ?')
    ->execute([100, 2]);
    $pdo->commit();                     // สำเร็จทั้งหมด → ยืนยัน
} catch (Exception $e) {
    $pdo->rollBack();                   // มี error → ย้อนทั้งหมด
}
```

6. สรุป Cheat Sheet ขั้นสูง

```
-- FOREIGN KEY + พฤติกรรมตอนลบแม่
FOREIGN KEY (fk_col) REFERENCES parent(id)
ON DELETE CASCADE;    -- หรือ RESTRICT / SET NULL

-- JOIN (เชื่อมหลายตารางต่อกันได้ เช่น M:N ผ่านตารางกลาง)
SELECT a.col, b.col
FROM tableA a
INNER JOIN tableB b ON a.fk = b.id;    -- LEFT JOIN ถ้าต้องการฝั่งซ้ายครบ

-- สรุปแบบกลุ่ม
SELECT กลุ่ม, SUM(ค่า) AS total
FROM ตาราง
GROUP BY กลุ่ม
HAVING total > X;

-- Transaction
START TRANSACTION;
... คำสั่งหลายอัน ...
SAVEPOINT p1;           -- (ถ้าต้องการ) ตั้งจุดเซฟ
ROLLBACK TO p1;         -- ย้อนเฉพาะบางส่วน
COMMIT;                -- หรือ ROLLBACK; เพื่อยกเลิกทั้งหมด
```

ความสัมพันธ์ 3 แบบ: 1:1 · 1:N (ฝั่งหลายเก็บ FK) · M:N (ต้องมีตารางกลาง)

Lab 2 · ฝึก SQL ขั้นสูง (JOIN / GROUP BY / Transaction)

เป้าหมาย

ฝึกเชื่อมตารางด้วย JOIN (รวมความสัมพันธ์ Many-to-Many), สรุปข้อมูลด้วย GROUP BY และทดลอง Transaction จริง

สิ่งที่ต้องเตรียม

- phpMyAdmin → ฐานข้อมูล `atsoft_day1`
- Import `sql/02_advanced_seed.sql` → ได้ตาราง `products`, `customers`, `orders`, `accounts`, `order_items`
(ไฟล์นี้สร้างตาราง `products` ให้ในตัว ไม่ต้อง import basic ก่อน)
- ตรวจว่ามีสินค้า 10 รายการ, ลูกค้า 4 คน, คำสั่งซื้อ 6 รายการ, รายการในคำสั่งซื้อ (`order_items`) 9 แถว, บัญชี 2 บัญชี

เขียนคำตอบในช่องว่าง แล้วเทียบ เฉลย Lab 2

ส่วนที่ 1: JOIN

ข้อ 1.1 แสดง ชื่อลูกค้า, สินค้า, จำนวน ของทุกคำสั่งซื้อ (ใช้ INNER JOIN)

ข้อ 1.2 แสดงคำสั่งซื้อทั้งหมดของลูกค้าชื่อ มานี รักเรียน

JOIN แล้วใส่ `WHERE c.name = 'มานี'`

ข้อ 1.3 แสดง ลูกค้าทุกคน พร้อมสินค้าที่สั่ง — รวมคนที่ยังไม่เคยสั่งด้วย (ใช้ LEFT JOIN)

ลูกค้าที่ไม่เคยสั่งจะเห็น product เป็น NULL

ส่วนที่ 2: Aggregate + GROUP BY

ข้อ 2.1 นับว่ามีคำสั่งซื้อทั้งหมดกี่รายการ

ข้อ 2.2 หว่าลูกค้าแต่ละคนสั่งสินค้า รวมกี่ชิ้น (แสดงชื่อลูกค้า + ยอดรวม)

```
GROUP BY c.name แล้ว SUM(o.amount)
```

ข้อ 2.3 คำนวณ ยอดเงินรวม ของแต่ละคำสั่งซื้อ (จำนวน × ราคา) และแสดงคู่กับชื่อลูกค้า

```
SELECT c.name, o.product, (o.amount * o.price) AS total ...
```

ข้อ 2.4 หายอดเงินรวมที่ลูกค้าแต่ละคนใช้จ่ายทั้งหมด เรียงจากมากไปน้อย

```
SUM(o.amount * o.price) + GROUP BY + ORDER BY ... DESC
```

ข้อ 2.5 แสดงเฉพาะลูกค้าที่ใช้จ่ายรวม เกิน 100 บาท (ใช้ HAVING)

ส่วนที่ 3: Transaction (ฝึกบนตาราง accounts)

ดูยอดเงินตั้งต้นก่อน:

```
SELECT * FROM accounts;  
-- บัญชี A = 1000, บัญชี B = 500
```

ข้อ 3.1 จำลองการโอนเงิน 200 บาท จากบัญชี A (id=1) ไปบัญชี B (id=2) ด้วย Transaction แบบยืนยัน (COMMIT)

ตรวจสอบผล: A ควรเหลือ 800, B ควรเป็น 700

ข้อ 3.2 ทดลองโอนอีก 9999 บาทจาก A ไป B แต่ครั้งนี้ให้ **ROLLBACK** (ยกเลิก) แล้วเช็คว่ายอดเงินไม่เปลี่ยน

ทำ START TRANSACTION → UPDATE 2 บรรทัด → SELECT ดูยอด (จะเห็นยอดเปลี่ยนชั่วคราว) → ROLLBACK → SELECT อีกครั้ง (ยอดกลับเหมือนเดิม)

ข้อ 3.3 จำลองการขายจริงด้วย Transaction: ลูกค้า id=2 ซื้อ น้ำดื่ม 600ml 3 ชิ้น

ให้ทำ 2 อย่างใน Transaction เดียว แล้ว **COMMIT**: (1) หักสต็อกในตาราง `products` ลง 3 และ (2) เพิ่มแถวใน `orders`

```
UPDATE products SET stock = stock - 3 WHERE name = 'น้ำดื่ม 600ml'; + INSERT INTO orders (... ) VALUES (... )
```

ข้อ 3.4 (SAVEPOINT) เริ่ม Transaction → ถอนเงิน A ออก 100 → ตั้ง **SAVEPOINT s1** → เพิ่มเงิน B 100 → ตั้ง **ROLLBACK TO s1** (ยกเลิกเฉพาะการเพิ่มเงิน B) → **COMMIT** แล้วเช็ค A ลดแต่ B เท่าเดิม

ส่วนที่ 4: Many-to-Many (ตาราง `order_items`)

ความสัมพันธ์ M:N: 1 คำสั่งซื้อมีหลายสินค้า / 1 สินค้าอยู่ในหลายคำสั่งซื้อ เชื่อมผ่านตารางกลาง `order_items`

ข้อ 4.1 แสดงรายการสินค้าในแต่ละคำสั่งซื้อ: `order_id`, ชื่อสินค้า, จำนวน (JOIN 3 ตาราง: `orders` → `order_items` → `products`)

ข้อ 4.2 คำสั่งซื้อใบที่ 1 (`order_id = 1`) มีสินค้าอะไรบ้าง ก็ขึ้น

ข้อ 4.3 คำนวน ยอดเงินรวมของแต่ละคำสั่งซื้อ = ผลรวมของ (จำนวน × ราคาต่อชิ้น)

```
JOIN 3 ตาราง + SUM(oi.quantity * p.price) + GROUP BY o.id
```

ข้อ 4.4 สินค้าแต่ละชิ้นถูกสั่งซื้อรวมกี่ชิ้น (จากทุกคำสั่งซื้อ) เรียงมากไปน้อย

```
GROUP BY p.name + SUM(oi.quantity)
```

โจทย์ท้าทาย

ข้อ 5.1 หาวาสินค้าใดถูกสั่งซื้อรวม (amount) มากที่สุด

```
GROUP BY product + ORDER BY SUM(amount) DESC LIMIT 1
```

ข้อ 5.2 นับจำนวนคำสั่งซื้อของลูกค้าแต่ละคน รวมถึงคนที่ไม่มี 0 รายการ

```
LEFT JOIN + COUNT(o.id) (นับ o.id เพื่อให้คนไม่มี order ได้ 0)
```

เช็คความเข้าใจ

- [] เขียน INNER JOIN และ LEFT JOIN ได้ และเข้าใจความต่าง
- [] JOIN 3 ตารางผ่านตารางกลาง (M:N) ได้
- [] ใช้ GROUP BY + SUM/COUNT สรุปข้อมูลได้
- [] ใช้ HAVING กรองผลที่สรุปแล้วได้
- [] เข้าใจว่า COMMIT ยืนยัน, ROLLBACK ยกเลิก และใช้ SAVEPOINT ย้อนบางส่วนได้

เฉลย Lab 1 · SQL พื้นฐาน

ลองทำเองให้สุดก่อนเปิดเฉลยนะครับ — คำตอบบางข้อทำได้หลายแบบ ถ้าผลลัพธ์ถูกก็ถือว่าใช้ได้

ส่วนที่ 1: READ (SELECT)

1.1

```
SELECT * FROM products;
```

1.2

```
SELECT name, price FROM products;
```

1.3

```
SELECT * FROM products WHERE category = 'เครื่องดื่ม';
```

1.4

```
SELECT * FROM products WHERE price > 20;
```

1.5

```
SELECT * FROM products WHERE stock < 50;
```

1.6

```
SELECT * FROM products WHERE name LIKE '%น้ำ%';
```

1.7

```
SELECT * FROM products ORDER BY price DESC;
```

1.8

```
SELECT * FROM products ORDER BY price DESC LIMIT 3;
```